



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Recorridos Inorden, Preorden, Postorden y BFS
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3 – “B”
Alumnos participantes:	Quispe Nasimba Daniela Alejandra
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Implementar y analizar los principales recorridos de árboles binarios utilizando C++ y Java, aplicando estructuras de datos dinámicas, recursividad y colas.

Específicos:

- Desarrollar programas capaces de ejecutar correctamente los recorridos Inorden, Preorden, Postorden y BFS, aplicando estructuras jerárquicas mediante árboles binarios y verificando el funcionamiento de cada recorrido en ambos lenguajes de programación.
- Identificar similitudes y diferencias entre ambos lenguajes en aspectos como manejo de memoria, sintaxis, uso de punteros, objetos, recursividad y estructuras dinámicas aplicadas a árboles binarios.
- Representar módulos de un sistema web mediante una estructura jerárquica tipo árbol binario, relacionando los recorridos con procesos reales como menús, navegación de módulos y procesamiento de información.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

➤ **Creación y organización del proyecto:**

Crear un repositorio en GitHub para desarrollar la práctica de recorridos de árboles binarios. Posteriormente, organizar el proyecto en carpetas separadas para la implementación en C++, Java y documentación, permitiendo mantener un trabajo ordenado y fácil de comprender.

➤ **Implementación de los recorridos del árbol binario**

Desarrollar programas en C++ y Java capaces de construir un árbol binario y ejecutar correctamente los recorridos Inorden, Preorden, Postorden y BFS. Para los recorridos DFS se debe utilizar recursividad, mientras que para BFS se debe aplicar una cola que permita recorrer el árbol nivel por nivel.

➤ **Modificación y análisis del árbol**

Agregar nuevos nodos al árbol binario inicial y ejecutar nuevamente los recorridos para analizar cómo cambia la salida en cada caso. Además, implementar funciones adicionales que permitan contar la cantidad total de nodos y hojas existentes dentro de la estructura.



➤ **Aplicación a un caso real y documentación**

Representar mediante un árbol binario los módulos de un sistema web relacionado con el proyecto final, explicando qué tipo de recorrido sería más adecuado en diferentes situaciones. Finalmente, documentar el desarrollo completo de la práctica mediante un informe con código comentado, resultados obtenidos y capturas de ejecución.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computadora portátil.
- IDE.
- C++.
- Visual Studio Code.
- Internet.
- GitHub.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

- Investigar el funcionamiento de los árboles binarios y los diferentes tipos de recorridos utilizados en estructuras de datos.
- Diseñar y construir un árbol binario utilizando nodos dinámicos en C++ y Java.
- Implementar los recorridos Inorden, Preorden, Postorden y BFS aplicando recursividad y colas.
- Ejecutar y verificar el funcionamiento correcto de cada recorrido en ambos lenguajes de programación.
- Modificar el árbol agregando nuevos nodos y analizar los cambios producidos en los recorridos.
- Implementar funciones adicionales para contar nodos y hojas del árbol binario.
- Comparar las diferencias entre los recorridos DFS y BFS mediante ejemplos prácticos.
- Aplicar la estructura de árbol binario a un caso real relacionado con un sistema web o proyecto final.
- Documentar el código fuente y registrar evidencias de ejecución para incluirlas en el informe y repositorio GitHub.

2.7 Resultados obtenidos

Link de GitHub:

<https://github.com/Daniela-Quispe/Recorridos-de-Arboles-Binarios>

Recorridos de Arboles Binarios

1. INTRODUCCION:



Un recorrido de árbol es un proceso sistemático para visitar todos los nodos de una estructura jerárquica. En árboles binarios se utilizan principalmente recorridos en profundidad: Inorden, Preorden y Postorden. Además, el recorrido BFS permite visitar nodos por niveles usando una cola.

Los árboles binarios son estructuras de datos dinámicas utilizadas para organizar información de manera jerárquica. Cada nodo puede tener un hijo izquierdo y un hijo derecho, permitiendo representar relaciones entre elementos.

Dentro de las estructuras de datos, los recorridos de árboles son fundamentales porque permiten visitar todos los nodos siguiendo diferentes estrategias. Los recorridos más utilizados son:

- Inorden
- Preorden
- Postorden
- BFS (Breadth First Search)

Los recorridos Inorden, Preorden y Postorden pertenecen a DFS (Depth First Search), mientras que BFS trabaja por niveles utilizando una cola.

En esta tarea se desarrollaron programas en C++ y Java para implementar los diferentes recorridos de árboles binarios, analizar su funcionamiento y aplicar los conceptos a un caso real relacionado con un sistema web.

2. MARCO TEORICO:

Arboles Binarios:

Un árbol binario es una estructura dinámica formada por nodos conectados entre sí. Cada nodo puede contener:

- Un dato o valor.
- Un hijo izquierdo.
- Un hijo derecho.

El primer nodo del árbol se denomina raíz y es el punto de inicio de toda la estructura. Los nodos que no tienen hijos se conocen como hojas.

Nodos en un Árbol Binario:

Los nodos representan cada elemento almacenado dentro del árbol. Cada nodo contiene información y referencias hacia otros nodos.

En C++ se utilizan punteros para conectar nodos dinámicamente, mientras que en Java se utilizan referencias a objetos.

La utilización de nodos permite que el árbol pueda crecer dinámicamente durante la ejecución del programa.

Recursividad:

La recursividad es una técnica de programación donde una función se llama a sí misma para resolver un problema de manera repetitiva.

Los recorridos DFS utilizan recursividad porque cada subárbol puede tratarse como un árbol independiente.

Recorridos de Árboles Binarios:

Un recorrido es el proceso mediante el cual se visitan todos los nodos del árbol siguiendo un orden específico.

Los recorridos permiten:

- Mostrar información.
- Buscar datos.
- Procesar estructuras jerárquicas.



- Organizar información.
- Existen dos tipos principales:
- DFS (Depth First Search)
 - BFS (Breadth First Search)

DFS (Depth First Search):

DFS significa “búsqueda en profundidad”. Este método recorre primero las ramas más profundas del árbol antes de regresar a otros nodos.

Los recorridos DFS son:

- Inorden
- Preorden
- Postorden

DFS generalmente utiliza:

- Recursividad
- Pila implícita del sistema

Recorrido Inorden:

El recorrido Inorden sigue el orden: Izquierda → Raíz → Derecha

Recorrido Preorden:

El recorrido Preorden sigue el orden: Raíz → Izquierda → Derecha

Recorrido Postorden:

El recorrido Postorden sigue el orden: Izquierda → Derecha → Raíz

BFS (Breadth First Search):

BFS significa “búsqueda en anchura”. Este recorrido visita los nodos nivel por nivel.

Árboles Binarios en C++:

En C++ los árboles binarios se implementan utilizando:

- estructuras (struct)
- punteros
- memoria dinámica (new)

Ejemplo:

Nodo izquierdo;*

Nodo derecho;*

Árboles Binarios en Java:

En Java los árboles binarios se implementan mediante:

- clases
- objetos
- referencias

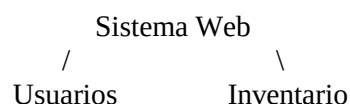
Ejemplo:

Nodo izquierdo;

Nodo derecho;

Aplicación de Árboles Binarios en Sistemas Reales:

En esta práctica se aplicó un árbol binario para representar módulos de un sistema web:





/ \ / \
Registrar Buscar Productos Reportes

Esto permite comprender cómo las estructuras jerárquicas pueden utilizarse para organizar módulos y procesos dentro de aplicaciones reales.

3. DESARROLLO DE LOS EJERCICIOS:

La práctica se desarrolló mediante la implementación de recorridos de árboles binarios en los lenguajes C++ y Java, utilizando estructuras dinámicas, recursividad y colas.

EJERCICIO 1:

Dado este árbol:

```
      10
     /  \
    5    15
   /\   /\
  2 7 12 20
```

Escriba manualmente:

- Preorden
- Inorden
- Postorden
- BFS

Construcción del Árbol Binario:

Inicialmente se creó un árbol binario utilizando nodos enlazados dinámicamente. Cada nodo almacena un valor y referencias hacia sus hijos izquierdo y derecho.

El árbol utilizado fue el siguiente:

```
      10
     /  \
    5    15
   /\   /\
  2 7 12 20
```

Donde:

- 10 representa la raíz principal.
- 5 y 15 corresponden al segundo nivel.
- 2, 7, 12 y 20 representan nodos hoja.

La construcción del árbol se realizó manualmente asignando cada hijo izquierdo y derecho en ambos lenguajes de programación.

Implementación de Recorridos DFS:

Posteriormente se implementaron los recorridos DFS utilizando funciones recursivas. Estos recorridos permiten explorar completamente una rama del árbol antes de pasar a otra.

Los recorridos implementados fueron:

- Preorden
- Inorden



- Postorden

Recorrido Preorden:

El recorrido Preorden sigue el orden: Raíz → Izquierda → Derecha

Proceso realizado:

1. Visitar la raíz 10.
2. Recorrer el subárbol izquierdo.
3. Recorrer el subárbol derecho.

Resultado obtenido: 10 5 2 7 15 12 20

Este recorrido permitió observar primero la estructura principal del árbol antes de recorrer sus niveles inferiores.

Recorrido Inorden:

El recorrido Inorden sigue el orden: Izquierda → Raíz → Derecha

Proceso realizado:

1. Recorrer completamente el subárbol izquierdo.
2. Visitar la raíz.
3. Recorrer el subárbol derecho.

Resultado obtenido: 2 5 7 10 12 15 20

Este recorrido mostró los valores ordenados de menor a mayor debido a la estructura del árbol binario.

Recorrido Postorden:

El recorrido Postorden sigue el orden: Izquierda → Derecha → Raíz

Proceso realizado:

1. Recorrer el subárbol izquierdo.
2. Recorrer el subárbol derecho.
3. Visitar la raíz al final.

Resultado obtenido: 2 7 5 12 20 15 10

Este recorrido permitió procesar primero los nodos hijos antes de trabajar con el nodo principal.

Implementación del Recorrido BFS:

Después de implementar DFS, se desarrolló el recorrido BFS utilizando una cola. BFS permite recorrer el árbol nivel por nivel.

Proceso realizado:

1. Insertar la raíz en la cola.
2. Extraer el nodo frontal.
3. Mostrar el valor.
4. Insertar los hijos izquierdo y derecho.
5. Repetir el proceso hasta vaciar la cola.

Resultado obtenido: 10 5 15 2 7 12 20

La utilización de la cola permitió mantener el orden correcto de recorrido por niveles.

EJERCICIO 2:



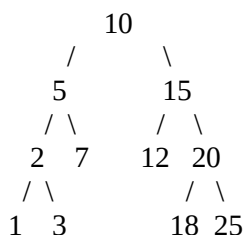
Modifique el árbol anterior agregando los nodos 1, 3, 18 y 25. Ejecute nuevamente los recorridos.

Modificación del Árbol Binario:

Posteriormente se agregaron nuevos nodos al árbol original para analizar cómo cambian los recorridos. Nuevos nodos agregados:

- 1
- 3
- 18
- 25

Árbol actualizado:



Con esta modificación se ejecutaron nuevamente los recorridos para verificar el funcionamiento correcto del programa.

Resultados Actualizados:

Preorden: 10 5 2 1 3 7 15 12 20 18 25

Inorden: 1 2 3 5 7 10 12 15 18 20 25

Postorden: 1 3 2 7 5 12 18 25 20 15 10

BFS: 10 5 15 2 7 12 20 1 3 18 25

Los resultados obtenidos demostraron que los recorridos funcionan correctamente incluso después de modificar la estructura del árbol.

Implementación de Funciones Adicionales:

Además de los recorridos, se implementaron funciones complementarias para fortalecer el manejo del árbol binario.

EJERCICIO 3:

Implemente una función que cuente la cantidad total de nodos del árbol.

Conteo Total de Nodos: Se implementó una función recursiva capaz de contar todos los nodos del árbol.

Proceso:

1. Verificar si el nodo es nulo.
2. Contar el nodo actual.
3. Contar recursivamente el subárbol izquierdo.
4. Contar recursivamente el subárbol derecho.

Resultado obtenido: Total de nodos: 11

EJERCICIO 4:

Implemente una función que cuente las hojas del árbol.



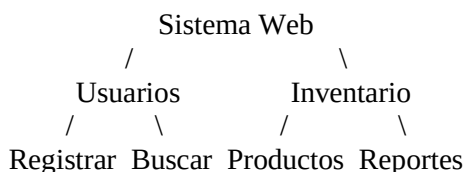
Conteo de Hojas: También se implementó una función para identificar y contar las hojas del árbol. Se consideró hoja a todo nodo sin hijos.

Hojas identificadas: 1 3 7 12 18 25

Resultado: 6 hojas

EJERCICIO 5 APLICADO AL PROYECTO FINAL:

Represente los módulos de un sistema web como un árbol Binario. Ejemplo:



Explique qué recorrido usaría para:

- Mostrar menú principal.
- Procesar primero los módulos internos.
- Mostrar módulos nivel por nivel.

Caso aplicado:

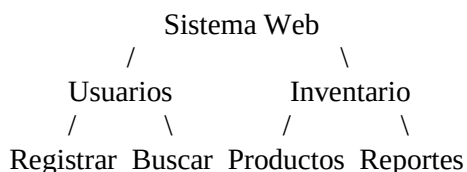
Una empresa desea organizar sus módulos de un sistema web mediante un árbol binario:

- Nodo raíz: Sistema principal
- Subárbol izquierdo: Gestión de usuarios
- Subárbol derecho: Gestión de inventario
- Nodos hoja: Operaciones específicas como registrar, buscar, actualizar y eliminar.

Aplicación al Caso Real:

Finalmente, se aplicó el árbol binario a un sistema web para representar módulos jerárquicos.

Estructura utilizada:



Aplicaciones de recorridos:

- Preorden: Mostrar el menú principal primero.
- Postorden: Procesar módulos secundarios antes del principal.
- BFS: Mostrar módulos nivel por nivel.

Comparación entre C++ y Java:

Durante la práctica se observó que ambos lenguajes permiten implementar árboles binarios de manera eficiente.

En C++:

- se utilizaron punteros,
- memoria dinámica,
- estructuras (struct).

En Java:



- se utilizaron objetos,
- clases,
- referencias automáticas.

Aunque la lógica de recorridos es similar, la sintaxis y manejo de memoria presentan diferencias importantes entre ambos lenguajes.

4. IMPLEMENTACION DE LOS EJERCICIOS EN C++:

EJERCICIO 1:

Recorrido Preorden:

```
void preorden(Nodo* raiz) {  
    if (raiz == NULL) return;  
    cout << raiz->dato << " ";  
    preorden(raiz->izquierdo);  
    preorden(raiz->derecho);  
}
```

Funcionamiento: La función realiza el recorrido Preorden utilizando recursividad. Primero visita la raíz, luego recorre el subárbol izquierdo y finalmente el subárbol derecho.

Recorrido Inorden:

```
void inorden(Nodo* raiz) {  
    if (raiz == NULL) return;  
    inorden(raiz->izquierdo);  
    cout << raiz->dato << " ";  
    inorden(raiz->derecho);  
}
```

Funcionamiento: La función recorre primero el subárbol izquierdo, después muestra la raíz y finalmente recorre el subárbol derecho.

Recorrido Postorden:

```
void postorden(Nodo* raiz) {  
    if (raiz == NULL) return;  
    postorden(raiz->izquierdo);  
    postorden(raiz->derecho);  
    cout << raiz->dato << " ";  
}
```

Funcionamiento: La función primero recorre los hijos izquierdo y derecho. La raíz se muestra al final del recorrido.

Recorrido BFS:

```
void bfs(Nodo* raiz) {  
    if (raiz == NULL) return;  
    queue<Nodo*> cola;  
    cola.push(raiz);
```



```
while (!cola.empty()) {  
    Nodo* actual = cola.front();  
    cola.pop();  
    cout << actual->dato << " ";  
  
    if (actual->izquierdo != NULL) cola.push(actual->izquierdo);  
    if (actual->derecho != NULL) cola.push(actual->derecho);  
}  
}
```

Funcionamiento: La función BFS utiliza una cola para recorrer el árbol nivel por nivel. Primero se inserta la raíz en la cola y posteriormente se recorren sus hijos en orden.

EJERCICIO 2:

Agregar Nuevos Nodos:

```
raiz->izquierdo->izquierdo->izquierdo = new Nodo(1);  
raiz->izquierdo->izquierdo->derecho = new Nodo(3);  
raiz->derecho->derecho->izquierdo = new Nodo(18);  
raiz->derecho->derecho->derecho = new Nodo(25);
```

Funcionamiento: Se agregan nuevos nodos al árbol original utilizando memoria dinámica. Esto permite modificar la estructura del árbol y ejecutar nuevamente los recorridos.

EJERCICIO 3:

Contar Nodos:

```
int contarNodos(Nodo* raiz) {  
    if (raiz == NULL) return 0;  
    return 1 + contarNodos(raiz->izquierdo) + contarNodos(raiz->derecho);  
}
```

Funcionamiento: La función utiliza recursividad para contar todos los nodos del árbol. Cuenta el nodo actual y suma los nodos de los subárboles izquierdo y derecho.

EJERCICIO 4:

Contar Hojas:

```
int contarHojas(Nodo* raiz) {  
    if (raiz == NULL) return 0;  
    if (raiz->izquierdo == NULL && raiz->derecho == NULL) return 1;  
    return contarHojas(raiz->izquierdo) + contarHojas(raiz->derecho);  
}
```

Funcionamiento: La función identifica los nodos que no tienen hijos y los considera hojas. Posteriormente cuenta todas las hojas existentes en el árbol.

EJERCICIO 5:

Sistema Web:



```
Nodo* raiz = new Nodo("Sistema Web");
raiz->izquierdo = new Nodo("Usuarios");
raiz->derecho = new Nodo("Inventario");
raiz->izquierdo->izquierdo = new Nodo("Registrar");
raiz->izquierdo->derecho = new Nodo("Buscar");
raiz->derecho->izquierdo = new Nodo("Productos");
raiz->derecho->derecho = new Nodo("Reportes");
```

Funcionamiento: El árbol representa módulos de un sistema web mediante una estructura jerárquica. Cada nodo corresponde a un módulo principal o secundario del sistema.

5. IMPLEMENTACION DE LOS EJERCICIOS EN JAVA:

EJERCICIO 1:

Recorrido Preorden:

```
static void preorden(Nodo raiz) {
    if (raiz == null) return;
    System.out.print(raiz.dato + " ");
    preorden(raiz.izquierdo);
    preorden(raiz.derecho);
}
```

Funcionamiento: La función recorre primero la raíz y posteriormente los subárboles izquierdo y derecho utilizando recursividad.

Recorrido Inorden:

```
static void inorden(Nodo raiz) {
    if (raiz == null) return;
    inorden(raiz.izquierdo);
    System.out.print(raiz.dato + " ");
    inorden(raiz.derecho);
}
```

Funcionamiento: La función recorre primero el hijo izquierdo, luego muestra la raíz y finalmente recorre el hijo derecho.

Recorrido Postorden:

```
static void postorden(Nodo raiz) {
    if (raiz == null) return;
    postorden(raiz.izquierdo);
    postorden(raiz.derecho);
    System.out.print(raiz.dato + " ");
}
```

Funcionamiento: La función procesa primero ambos subárboles y al final muestra la raíz principal.

Recorrido BFS:

```
static void bfs (Nodo raiz) {
    if (raiz == null) return;
    Queue<Nodo> cola = new LinkedList<>();
```



```
cola.add(raiz);
```

```
while (!cola.isEmpty()) {  
    Nodo actual = cola.poll();  
    System.out.print(actual.dato + " ");  
    if (actual.izquierdo != null) cola.add(actual.izquierdo);  
    if (actual.derecho != null) cola.add(actual.derecho);  
}  
}
```

Funcionamiento: La función BFS utiliza una cola para recorrer los nodos por niveles. Los nodos se almacenan temporalmente hasta completar el recorrido del árbol.

EJERCICIO 2:

Agregar Nuevos Nodos:

```
raiz.izquierdo.izquierdo.izquierdo = new Nodo(1);  
raiz.izquierdo.izquierdo.derecho = new Nodo(3);  
raiz.derecho.derecho.izquierdo = new Nodo(18);  
raiz.derecho.derecho.derecho = new Nodo(25);
```

Funcionamiento: Se agregan nuevos nodos al árbol binario original para modificar su estructura. Cada nuevo nodo se conecta mediante referencias hacia los hijos izquierdo y derecho. Después de insertar los nodos, los recorridos se ejecutan nuevamente para observar los cambios producidos en el árbol.

EJERCICIO 3:

Contar Nodos:

```
static int contarNodos(Nodo raiz) {  
    if (raiz == null) return 0;  
    return 1 + contarNodos(raiz.izquierdo) + contarNodos(raiz.derecho);  
}
```

Funcionamiento: La función utiliza recursividad para recorrer todo el árbol binario. Cuenta el nodo actual y suma la cantidad de nodos encontrados en el subárbol izquierdo y derecho. El resultado corresponde al número total de nodos existentes en el árbol.

EJERCICIO 4:

Contar Hojas:

```
static int contarHojas(Nodo raiz) {  
    if (raiz == null) return 0;  
    if (raiz.izquierdo == null && raiz.derecho == null) return 1;  
    return contarHojas(raiz.izquierdo) + contarHojas(raiz.derecho);  
}
```

Funcionamiento: La función identifica los nodos que no tienen hijos y los considera hojas. Posteriormente recorre el árbol utilizando recursividad para calcular la cantidad total de hojas.

EJERCICIO 5:



Sistema Web:

```
Nodo raiz = new Nodo("Sistema Web");
raiz.izquierdo = new Nodo("Usuarios");
raiz.derecho = new Nodo("Inventario");
raiz.izquierdo.izquierdo = new Nodo("Registrar");
raiz.izquierdo.derecho = new Nodo("Buscar");
raiz.derecho.izquierdo = new Nodo("Productos");
raiz.derecho.derecho = new Nodo("Reportes");
```

Funcionamiento: El árbol binario representa la estructura jerárquica de un sistema web. La raíz representa el sistema principal y los demás nodos corresponden a módulos secundarios. Esta representación permite aplicar los recorridos a situaciones reales dentro del desarrollo de software.

6. CAPTURAS EN C++:

Se incluyen capturas del funcionamiento del programa:

a) Ejercicio 1 En C++:

```
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20

-----
Process exited after 0.06136 seconds with return value 0
Presione una tecla para continuar . . .
```

b) Ejercicio 2 En C++:

```
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 1 3 7 15 12 20 18 25
Inorden: 1 2 3 5 7 10 12 15 18 20 25
Postorden: 1 3 2 7 5 12 18 25 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 18 25

-----
Process exited after 0.04517 seconds with return value 0
Presione una tecla para continuar . . .
```

c) Ejercicio 3 En C++:

```
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 1 3 7 15 12 20 18 25
Inorden: 1 2 3 5 7 10 12 15 18 20 25
Postorden: 1 3 2 7 5 12 18 25 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 18 25
Total de nodos: 11

-----
Process exited after 0.03649 seconds with return value 0
Presione una tecla para continuar . . .
```

d) Ejercicio 4 En C++:

```
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 1 3 7 15 12 20 18 25
Inorden: 1 2 3 5 7 10 12 15 18 20 25
Postorden: 1 3 2 7 5 12 18 25 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 18 25
Total de hojas: 6

-----
Process exited after 0.05425 seconds with return value 0
Presione una tecla para continuar . . .
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



e) Ejercicio 5 En C++ (Menú):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: |
```

f) Ejercicio 5 En C++ (Opción 1):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 1

PREORDEN: Sistema Web -> Usuarios -> Registrar -> Buscar -> Inventario -> Productos -> Reportes
```

g) Ejercicio 5 En C++ (Opción 2):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 2

INORDEN: Registrar -> Usuarios -> Buscar -> Sistema Web -> Productos -> Inventario -> Reportes
```

h) Ejercicio 5 En C++ (Opción 3):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 3

POSTORDEN: Registrar -> Buscar -> Usuarios -> Productos -> Reportes -> Inventario -> Sistema Web
```

i) Ejercicio 5 En C++ (Opción 4):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 4

BFS: Sistema Web -> Usuarios -> Inventario -> Registrar -> Buscar -> Productos -> Reportes
```

j) Ejercicio 5 En C++ (Opción 5):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 5

Cerrando sistema de arboles binarios...

-----
Process exited after 223.1 seconds with return value 0
Presione una tecla para continuar . . .
```



7. CAPTURAS EN JAVA:

Se incluyen capturas del funcionamiento del programa:

a) Ejercicio 1 En Java:

```
HP@DESKTOP-ST1S8TI MINGW64 ~/Downloads/Java (master)
$ javac *.java

HP@DESKTOP-ST1S8TI MINGW64 ~/Downloads/Java (master)
$ java Main
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
```

b) Ejercicio 2 En Java:

```
HP@DESKTOP-ST1S8TI MINGW64 ~/Downloads/Java (master)
$ java Main2
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 1 3 7 15 12 20 18 25
Inorden: 1 2 3 5 7 10 12 15 18 20 25
Postorden: 1 3 2 7 5 12 18 25 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 18 25
```

c) Ejercicio 3 En Java:

```
HP@DESKTOP-ST1S8TI MINGW64 ~/Downloads/Java (master)
$ java Main3
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 1 3 7 15 12 20 18 25
Inorden: 1 2 3 5 7 10 12 15 18 20 25
Postorden: 1 3 2 7 5 12 18 25 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 18 25
Total de nodos: 11
```

d) Ejercicio 4 En Java:

```
HP@DESKTOP-ST1S8TI MINGW64 ~/Downloads/Java (master)
$ java Main4
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 1 3 7 15 12 20 18 25
Inorden: 1 2 3 5 7 10 12 15 18 20 25
Postorden: 1 3 2 7 5 12 18 25 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 18 25
Total de hojas: 6
```

e) Ejercicio 5 En Java (Menú):

```
HP@DESKTOP-ST1S8TI MINGW64 ~/Downloads/Java (master)
$ java Main5

===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: |
```

f) Ejercicio 5 En Java (Opción 1):

```
HP@DESKTOP-ST1S8TI MINGW64 ~/Downloads/Java (master)
$ java Main5

===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 1

PREORDEN: Sistema Web -> Usuarios -> Registrar -> Buscar -> Inventario -> Productos -> Reportes
```




g) Ejercicio 5 En Java (Opción 2):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 2

INORDEN: Registrar -> Usuarios -> Buscar -> Sistema Web -> Productos -> Inventario -> Reportes
```

h) Ejercicio 5 En Java (Opción 3):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 3

POSTORDEN: Registrar -> Buscar -> Usuarios -> Productos -> Reportes -> Inventario -> Sistema Web
```

i) Ejercicio 5 En Java (Opción 4):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 4

BFS: Sistema Web -> Usuarios -> Inventario -> Registrar -> Buscar -> Productos -> Reportes
```

j) Ejercicio 5 En Java (Opción 5):

```
===== MENU PRINCIPAL DE UN ARBOL BINARIO =====
1. Preorden (Mostrar estructura del sistema)
2. Inorden (Recorrido ordenado)
3. Postorden (Ejecutar modulos internos primero)
4. BFS (Recorrido jerarquico por niveles)
5. Salir
Seleccione: 5

Cerrando sistema de arboles binarios...

HP@DESKTOP-ST1S8TI MINGW64 ~/Downloads/Java (master)
$ |
```

8. DOCUMENTACION DE GITHUB E IA:

GitHub:

Durante el desarrollo de la práctica se utilizó GitHub como plataforma de control de versiones y almacenamiento del proyecto, permitiendo organizar el código fuente, la documentación y las evidencias de ejecución de manera estructurada. El repositorio facilitó el acceso a los archivos desarrollados en C++ y Java, además de permitir mantener un respaldo del trabajo realizado.

El enlace del repositorio utilizado en la tarea fue:

<https://github.com/Daniela-Quispe/Recorridos-de-Arboles-Binarios>

IA:

Se utilizó Inteligencia Artificial como herramienta de apoyo académico durante el desarrollo de la práctica. La IA colaboró en la explicación de conceptos relacionados con árboles binarios, recorridos DFS y BFS, recursividad y estructuras dinámicas. También brindó apoyo parcial en la generación y corrección de fragmentos de código en C++ y Java, así como en la organización y redacción del informe técnico.

2.8 Habilidades blandas empleadas en la práctica



- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se logró implementar correctamente los recorridos Inorden, Preorden, Postorden y BFS en los lenguajes C++ y Java, permitiendo comprender el funcionamiento de los árboles binarios y la forma en que se recorren sus nodos mediante diferentes estrategias.
- La práctica permitió aplicar recursividad en los recorridos DFS, demostrando cómo una función puede recorrer estructuras jerárquicas de manera eficiente mediante llamadas recursivas sobre los subárboles izquierdo y derecho.
- Se comprobó la importancia del uso de colas en el recorrido BFS, ya que esta estructura permite recorrer el árbol nivel por nivel manteniendo el orden correcto de visita de los nodos.
- Mediante la comparación entre C++ y Java se identificaron diferencias importantes en el manejo de memoria y estructuras dinámicas. En C++ se trabajó con punteros y memoria dinámica, mientras que en Java se utilizaron clases, objetos y referencias administradas automáticamente.
- La implementación de funciones adicionales para contar nodos y hojas permitió fortalecer el manejo de árboles binarios y comprender mejor la manipulación de estructuras dinámicas mediante recursividad.
- La representación de módulos de un sistema web mediante un árbol binario permitió relacionar los conceptos teóricos con una aplicación real, demostrando que los árboles son útiles para organizar información jerárquica dentro del desarrollo de software.

2.10 Recomendaciones

- Continuar practicando la implementación de árboles binarios y diferentes tipos de recorridos para fortalecer la comprensión de estructuras de datos dinámicas y algoritmos recursivos.
- Realizar pruebas agregando nuevos nodos y modificando la estructura del árbol para analizar cómo cambian los recorridos y el comportamiento del programa.
- Aplicar los árboles binarios en casos reales relacionados con sistemas web, organización de información y procesamiento jerárquico de datos para fortalecer la relación entre teoría y práctica.

2.11 Referencias bibliográficas

- [1] Enjoy Algorithms, “Binary Tree Traversals: Preorder, Inorder and Postorder,” 2023. [En línea]. Disponible en: <https://www.enjoyalgorithms.com/blog/binary-tree-traversals-preorder-inorder-postorder> [Accedido: 07-may-2026].
- [2] GeeksforGeeks, “Inorder Traversal of Binary Tree,” 2025. [En línea]. Disponible en: <https://www.geeksforgeeks.org/dsa/inorder-traversal-of-binary-tree/> [Accedido: 07-may-2026].
- [3] Interview Cake, “Binary Tree,” 2024. [En línea]. Disponible en: <https://www.interviewcake.com/concept/java/binary-tree> [Accedido: 07-may-2026].



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



[4] Wikipedia, “Árbol binario,” 2025. [En línea]. Disponible en: https://es.wikipedia.org/wiki/%C3%81rbol_binario [Accedido: 07-may-2026].

2.12 Anexos

GitHub Personal:

